The MABEL Teleoperation Protocol: How the iPhone, Vision Pro, and Keyboard Clients Drive the Simulated (and Real) Robot

MABEL project — `server/` + client apps

November 4, 2026

Abstract

This document specifies the *communication contract* by which every MABEL teleoperation client — the iPhone app, the Apple Vision Pro app, and (over a parallel path) the ROS keyboard node — drives the MuJoCo-simulated robot, with no change required to reach the real robot. The apps speak a single, transport-light protocol: a typed two-layer JSON `WireEnvelope` over *one* WebSocket (`ws://<host>:9090/teleop`) carries operator intent up, and *plain HTTP MJPEG* (`http://<host>:8080/camera/*`) carries the robot's cameras down. The schema is mirrored byte-for-byte in three languages (Swift on both apps, Python on the server), so a client and the bridge always agree. We document the envelope and every message type, the principal `TeleopFrame` (head pose + per-hand $SE(3)$ joints, or a navigation joystick), how each client *bridges* into it (the iPhone `WebSocketLink` at 20 Hz, the Vision Pro `PoseStreamCoordinator` at 60 Hz), Bonjour discovery and the connect/keepalive/auto-reconnect state machine, and the MJPEG video framing. We close by contrasting this app path with the ROS topic path the keyboard teleop uses to reach the *same* simulator.

Keywords: teleoperation protocol, WebSocket, JSON envelope, MJPEG, Apple Vision Pro, ARKit hand tracking, Bonjour/mDNS, ROS 2, hardware-agnostic transport.

1 Overview

A MABEL operator can be wearing a Vision Pro, holding an iPhone, or sitting at a keyboard, and in every case the same simulated robot must move. The system achieves this with a deliberately thin *communication contract* rather than a custom binary protocol: clients exchange typed JSON over a standard WebSocket and pull video as standard MJPEG. Two properties fall out of that choice. First, the schema is trivially mirrored in Swift and Python, so the apps and the server cannot silently disagree (there is even a Swift `WireCodecTests` suite checking round-trips). Second, because the contract names *robot intent*, not simulator internals, the identical messages drive the real robot once a hardware-side process answers the same endpoints.

This document covers the **app path** (WebSocket + MJPEG) end-to-end (Fig. 1). The server side of that path — the bridge that turns these messages into physics — is documented separately in the companion `server.pdf`; here we focus on the wire format and the *client* bridges. Section 7 contrasts the second, independent **keyboard path**, which reaches the same sim through ROS 2 topics instead.

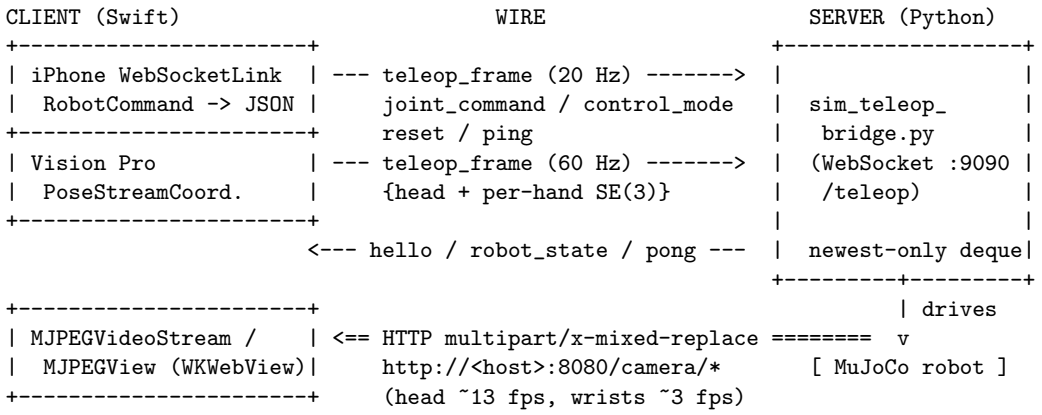


Figure 1: The app communication path. One WebSocket carries the typed command envelopes up; one HTTP MJPEG connection per camera carries video down. The iPhone and Vision Pro differ only in how they *fill* a `TeleopFrame` and at what rate.

Table 1: Message types (`WireMessageType`). All are carried in the same envelope.

Type	Direction	Payload / meaning
`teleop_frame`	client → server	the principal frame: head pose, optional hands, mode, nav joystick (§2.2)
`joint_command`	client → server	{`joints`:{`id`:`rad`}} direct joint targets (iPhone Body tab / WBC)
`control_mode`	client → server	{`mode`}: navigation, gravity_comp, compliance, or joints
`reset`	client → server	home the simulation (`goHome`/`resetSim`)
`hello`	server → client	handshake on connect; carries {`server`, `version`} identity
`robot_state`	server → client	telemetry: battery, `jointPositions`, `latencyMs`
`ping` / `pong`	either	application-level liveness
`error`	server → client	diagnostic string

2 Method: the wire protocol

2.1 The envelope

Every message is a two-layer JSON object,

{ "type": <WireMessageType>, "payload": { ... } },

defined once and mirrored in `wire_protocol.py` (Python, server) and `WireEnvelope.swift` / `PoseModels.swift` (Swift, both apps). Field names are the Swift `Codable` camelCase defaults, so no name translation is needed across the wire. The envelope is intentionally minimal: the Swift decoder keeps the `payload` as raw `Data` (via an internal `AnyCodable`) and only decodes it to a concrete type once `type` says it is worth decoding, which keeps inbound handling allocation-light. The Python side is stdlib-`json` only, so the same decoder can be imported into a ROS 2 node without extra dependencies.

2.2 The `TeleopFrame`

The frame that actually moves the robot carries a monotonically increasing `sequence`, a `timestamp`, a `HeadPose`, optional `leftHand` and `rightHand`, a `mode` (`Arms & Hands` or `Base Driving`), and an optional `navJoystick`. Every pose is a **row-major** homogeneous transform serialised as 16 floats,

$$T = \begin{bmatrix} R & p \\ \mathbf{0}^\top & 1 \end{bmatrix} \in SE(3), \qquad R \in SO(3),\ p \in \mathbb{R}^3,$$

expressed in the ARKit world frame (x right, y up, $-z$ forward). A `HandPose` is an `anchorTransform` plus a list of named joint `localTransform`s; the world pose of joint j is

$$T_j^{\text{world}} \;=\; T_{\text{anchor}}\, T_j^{\text{local}}, \tag{1}$$

exactly the composition the server-side adapter inverts to recover wrist/thumb/index tips for retargeting (see `server.pdf`, §Method). The `navJoystick` fields $(\ell_x, \ell_y, r_x, r_y)$ are normalised to $[-1, 1]$: left stick translates the base (ℓ_y forward, ℓ_x strafe), right stick yaws (r_x) and lifts (r_y). A client therefore drives manipulation by populating the hands, or base driving by populating the joystick — the same message either way.

2.3 Telemetry back

The server greets each connection with `hello` so the client learns *what it is driving* (e.g. `"mabel_sim_bridge"` vs. a hardware identity) and then streams `robot_state` carrying battery, a `jointPositions` map, and a round-trip `latencyMs`. The iPhone surfaces all three in its UI; both apps treat the first inbound message as proof-of-life that promotes the connection to `connected`.

3 Method: the client bridges

Both apps reduce to the same job — emit `TeleopFrame`s — but differ in source and rate.

iPhone (`WebSocketLink.swift`). The app is a *pure input device*: the UI issues abstract `RobotCommand`s (`twist`, `liftRate`, `jointTarget`, `controlMode`, `estop`, `goHome`, ...) to a `RobotLink` protocol, and `WebSocketLink` encodes them to the wire. A repeating 20 Hz timer calls `pushFrame()`, which sends a `Base Driving` `teleop_frame` carrying the latest joystick state, and flushes a `joint_command` only when joint targets change. Stick values are mapped to the sim's convention at the edge (forward and yaw are negated; values clamped to $[-1, 1]$), so the server receives canonical intent. Because the robot models live server-side, the phone holds almost no state.

Vision Pro (`PoseStreamCoordinator.swift`). ARKit delivers head and 27-joint hand snapshots at $\sim$ 90 Hz; a detached task keeps the newest hand snapshot in a `LatestHandSnapshot`, while a second timer-driven task emits `TeleopFrame`s at `transmissionRateHz` (default 60 Hz). In windowed mode the hands are `nil` (head-only); in the immersive space they are populated from the latest snapshot. Both tasks run off the main actor so ARKit $\rightarrow$ encode $\rightarrow$ network never blocks on SwiftUI or video decode.

Discovery and connection lifecycle. Clients need not know the Mac's IP: the server advertises `_mabel-teleop._tcp` over Bonjour (`run.sh`), and `ServerDiscovery.swift` browses for it and resolves each to a `.local` host. The iPhone link then runs a small resilience state machine: an 8 s handshake *watchdog*, a 5 s application ping for keepalive (a failed ping does *not* tear down — that was a source of reconnect flapping; only `receive()` failures signal a real drop), and an auto-reconnect with gentle linear backoff capped at 4 s, persisting until the operator explicitly disconnects. Transient `send` errors are swallowed: one dropped frame out of a 20 Hz stream must never kill the session.

4 Method: the video channel

Video is deliberately *not* on the WebSocket. Two transports carry it, sharing the same JPEG frames server-side:

HTTP MJPEG (`:8080`, TCP) — universal. Each camera is a standalone HTTP `multipart/x-mixed-replace` (MJPEG) stream at `http://<host>:8080/camera/<name>/stream.mjpg`. The Vision Pro pulls each into a `WKWebView <img>` (and, for in-app textures, `MJPEGVideoStream.swift` parses the multipart boundaries using the per-part `Content-Length` header for frame-exact splitting); the iPhone shows them in an `MJPEGView`. Each HTTP subscriber holds a one-slot queue server-side, so a slow client drops stale frames rather than accumulating latency. Keeping video off the command socket means a heavy or stalled video decode can never delay the 20–60 Hz command stream.

Per-client bitrate (slow links). Over a slow TCP link (MJPEG on a weak Wi-Fi or Tailscale path) a full-resolution head frame ($\approx$ 50 kB at 960×720 q85) is too big: it stalls and queues. A client therefore asks for a lighter frame via query params `?q=<10..95>&scale=<0.1..1.0>`; the server decodes the freshest full-res JPEG and re-encodes it downscaled *per client*, off the event loop. A client on a slow link appends `q=50&scale=0.5` to the head streams (`main`, `head_right`) and `q=50` to the wrists; a viewer that sends no params keeps the crisp full frame untouched. Measured: the head frame drops $\approx 50 \rightarrow 10.5$ kB ($\approx 4.8\times$, to 480×360), i.e. $\approx$ 1 Mbps per mono eye — comfortably within a slow uplink — while two clients (full + shrunk) on the same stream both held $\approx$ 30 fps.

Latest-frame-wins UDP (`:9091`) — low-latency primary for the headset. TCP's reliability is wrong for teleop video: one lost segment stalls the whole MJPEG stream until the retransmit lands (head-of-line blocking, $\geq$ one RTO) and queues stale frames behind it. `udp_video.py` therefore serves the same JPEGs over UDP with *drop-don't-queue* semantics. The client subscribes by sending a JSON datagram `{"sub": ["main", "wrist_left", ...]}` every 1 s (it doubles as a keepalive; silent clients expire after 3 s) and receives a stream-name→id map back. Each JPEG then arrives as $\lceil n/1182 \rceil$ datagrams of $\leq$1196 B — sized to clear a Tailscale/WireGuard tunnel (MTU 1280) without IP fragmentation, since a fragmented datagram is lost if any fragment is — with a 14-byte big-endian header:

```
"MV" | ver u8 | stream u8 | frame_seq u16 | chunk u16 | chunks u16 | ts_ms u32
```

The receiver (`UDPVideoReceiver.swift`) reassembles per stream; a datagram of a *newer* frame (wrap-aware: forward $u16$ distance $< 2^{15}$) evicts any incomplete older frame, so loss can never make the display wait — a lost chunk costs exactly one frame, and the next renders on time. A JPEG is handed to the decoder *only* once every chunk has arrived (the per-stream bound is $\leq$4096 chunks $\approx$ 4.6 MB, a corrupt-header guard rather than a frame-size limit — the earlier 256-chunk bound silently dropped any frame over $\approx$ 295 kB, capping head resolution), so a partial bitstream is never

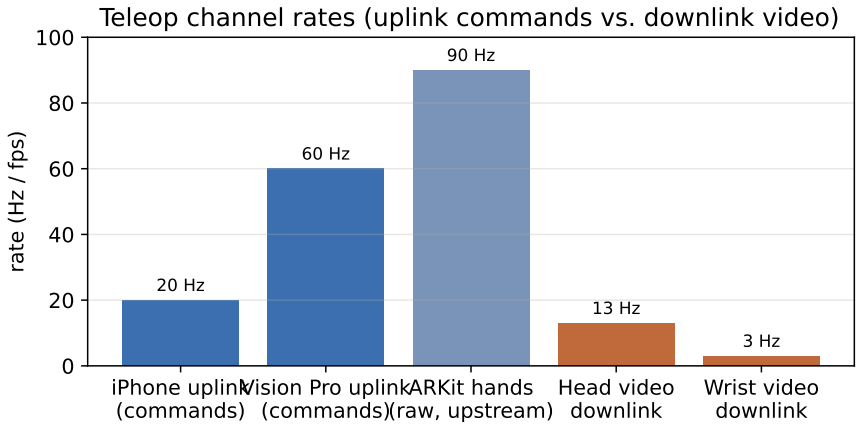


Figure 2: Configured teleop channel rates. Uplink command streams (iPhone 20 Hz, Vision Pro 60 Hz; ARKit samples hands at ∼90 Hz upstream of the rate limiter) versus measured downlink video (head ≈13 fps, wrist ≈3 fps).

shown. On the Vision Pro the decoded frame is uploaded to the curved head panel as a *new* `TextureResource` (with `.allocateAndGenerateAll` mipmaps) whose reference is swapped into the material atomically, never `replace(withImage:)` in place. The in-place call caused two artifacts at once: (i) the rasterizer could sample a half-written level 0, a moving zig-zag *tear*; and (ii) `replace` rewrites only level 0, so the coarse mip levels kept the first frame — and because the cylindrical panel samples trilinearly, the minified (far/edge) regions blended live detail with stale coarse mips and *crawled/shimmered* on any detailed content (seen on both the sim render and the real camera). Rebuilding the texture per frame double-buffers the upload (in-flight renders keep the previous frame) *and* regenerates the whole mip chain, so the head view is tear- and shimmer-free; the panel's image nodes are pinned to `filtertype="linear"` for crisp bilinear magnification. This is what lets the head stream at 1280×960 without the earlier moiré. The app's `VideoFeed` subscribes over UDP first and falls back to MJPEG automatically when no datagrams arrive within 2.5 s (UDP-hostile networks), reverting when they resume. The server advertises the port in the `hello` (`video.udpPort`); both camera-rendering processes install the sender via a `PushCapture.on_push` tee, so MJPEG clients (browser console, iOS app) are untouched. Measured behaviour — clean-link equivalence with MJPEG and flat delivered-frame latency under 1–10% datagram loss — is quantified in the Vision Pro app document (`vision_pro/docs/visionpro_app.pdf`, Revision 4).

5 Experiments

The protocol's measurable quantities are its channel rates; Fig. 2 plots the configured uplink rates and the measured downlink video rates on one axis (regenerated by `scripts/make_doc_figures.py`; the camera numbers are the measured values from `server.pdf`). The asymmetry is the design: a fast, light command uplink and a heavier, platform-limited video downlink, decoupled onto separate connections. The bridge consumes *newest-only*, so the effective control latency is set by the uplink period (16.7 ms at 60 Hz, 50 ms at 20 Hz) plus one physics substep, not by any queue depth.

Table 2: Protocol parameters (from the client and server source).

Parameter	Value	Source
Command transport	WebSocket `ws://<host>:9090/teleop`	`MabelConfig.plist`, `config.py`
Video transport (primary)	UDP latest-frame-wins `:9091`	`udp_video.py`, `UDPVideoReceiver.swift`
Video transport (fallback)	HTTP MJPEG `:8080/camera/*`	`MJPEGVideoStream.swift`
UDP datagram / payload size	$\leq$1196 B / 1182 B	`udp_video.py` (Tailscale MTU 1280)
UDP subscribe / expiry	1 s keepalive / 3 s TTL	`udp_video.py`
MJPEG fallback trigger	no UDP datagrams for 2.5 s	`UDPVideoReceiver.swift`
iPhone uplink rate	20 Hz	`WebSocketLink.swift`
Vision Pro uplink rate	60 Hz (`transmissionRateHz`)	`AppConfiguration.swift`
ARKit hand sample rate	$\sim$90 Hz	`PoseStreamCoordinator.swift`
Pose encoding	row-major 4×4, 16 floats, ARKit frame	`wire_protocol.py`
Server frame-consume policy	newest-only (1-slot deque)	`sim_teleop_bridge.py`
Keepalive ping	5 s (non-fatal)	`WebSocketLink.swift`
Handshake watchdog	8 s	`WebSocketLink.swift`
Reconnect backoff	$0.4k$ s, capped 4 s	`WebSocketLink.swift`
Discovery	Bonjour `_mabel-teleop._tcp`	`ServerDiscovery.swift`, `run.sh`

6 Implementation and Usage

Bring the server up (one command, prints the LAN IP and advertises Bonjour):

```
cd server && ./run.sh            # WebSocket :9090/teleop  +  MJPEG :8080/camera/*
```

iPhone. MABEL app ▷ Robot tab ▷ Connection: pick the auto-discovered server (or type the Mac's LAN IP) ▷ Connect, allow *Local Network*. The Drive tab in landscape shows the live cameras and drives the base. Host `demo`/blank runs the on-device `DemoLink` simulator instead.

Vision Pro. Settings ▷ Network ▷ Robot Host = the Mac's LAN IP (*not* `127.0.0.1`), Apply & Reconnect; grant Local Network, Hand Tracking, World Sensing; Start Teleop, then Enter Immersive.

Adding a client / message. Implement the `RobotLink` protocol (or its equivalent), encode the envelope of Table 1, and send to `/teleop`. New message types are added in one enum on each side (`WireMessageType`); unknown types are rejected at decode, so forward-compatibility is explicit.

7 The keyboard path (ROS), for contrast

The keyboard teleop reaches the *same* MuJoCo robot by a completely separate mechanism: it is a ROS 2 node (`mabel_ws`, `mabel_teleop/keyboard_teleop`) that publishes standard ROS topics — `/mabel_cmd` (`mabel_interfaces/MabelSwerveCmd`: v_x, v_y, ω, lift, e-stop), `/arms/command`, `/body/command`, `/<side>_hand/command` (`sensor_msgs/JointState`) — which a MuJoCo↔ROS bridge node (`mabel_sim/mujoco_node.py`) subscribes to and applies, publishing `/joint_states` and `/odom` back. Crucially, the *real* hardware nodes subscribe and publish those identical topics, so `ros2 launch mabel_bringup robot.launch.py use_sim:=true|false` swaps sim for hardware transparently. The two paths thus differ in transport (app WebSocket vs. ROS DDS) and in audience (a mixed-reality operator vs. a developer at a terminal or an autonomy stack), but both ultimately command the same robot; they are not bridged to each other today.

8 Limitations

- **No reliability layer on commands.** The uplink is fire-and-forget with newest-only consumption; a lost frame is invisible by design (the next one supersedes it). There is no sequencing-gap detection or retransmission.
- **JSON, not binary.** Human-readable and trivially cross-language, but a full two-hand frame is a few kB of text at 60 Hz; ample on Wi-Fi/LAN, not tuned for a constrained link.
- **App path and ROS path are disjoint.** A Vision Pro frame does not reach the ROS hardware nodes; bridging the WebSocket into ROS topics is future work.
- **LAN-only assumptions.** Bonjour discovery and `.local` hosts assume a shared subnet; remote operation uses explicit hosts (e.g. a Tailscale address).
- **Video downlink is platform-bound.** Head ≈13 fps is a macOS render limit, not a protocol limit (see `server.pdf`).

References

[1] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, IETF, 2011.

[2] "Motion JPEG over HTTP (`multipart/x-mixed-replace`)," informal web standard, Netscape, 1995.

[3] Apple Inc., "ARKit hand and world tracking (visionOS)," Developer Documentation, 2024.

[4] S. Cheshire and M. Krochmal, "DNS-Based Service Discovery," RFC 6763, IETF, 2013.

[5] MABEL project, "The MABEL Teleoperation Server" (`server/docs/server.pdf`).